



Understanding and Improving Human-AI Collaboration in Systems Engineering through Simulation

Ray Madachy
Naval Postgraduate School

INCOSE International Symposium 2026
Yokohama, Japan | June 13–18, 2026



Motivation

- Current LLM usage in engineering is largely ad hoc at individual and team levels.
- LLM integration is often informal with little measurement of effort or success.
- Organizations lack visibility into time consumption and reliability.
- Need for a repeatable, measurable engineering process for generative AI.
- The Personal Generation Process (PGP) model treats human–AI interaction as a controllable engineering process with feedback rather than informal craft.

Wa Connection

- *Wa* denotes harmony, balance, and collaborative unity. This work aspires not merely to use AI as a tool, but to achieve a disciplined, harmonious integration of human judgment and AI capability within engineering work.
- The PGP model embodies this theme directly. It treats human–AI interaction as a structured, measurable process in which the human practitioner remains both the operator and the reflective analyst of their own collaboration.
- Introspective simulation makes the dynamics of human–AI *wa* visible and quantifiable with controllable process parameters rather than invisible.
- PGP operationalizes awareness of one's role within a larger harmonious system through empirical data collection, process modeling, and continuous improvement.
- The result is *engineered harmony*: AI augments human throughput and quality while the human practitioner retains full accountability, visibility, and control.

Process Simulation for Human–AI Collaboration

- Human–AI collaboration should be treated as a process improvement problem, not just a tool evaluation problem.
- Simulation provides holistic process perspective using simulation of interrelated factors as a quantitative basis for deciding how to optimize AI value.
- Framework for analyzing non-obvious behaviors in development lifecycles. Leverages process feedback loop dynamics.
- Models feedback between user and AI, both within chats and across chats via learning and improvement using metrics. Accounts for corrective action cycles.
- Can model human–AI interaction as a controllable discrete event process with continuous model elements.

PSP and TSP Methodologies (1/2)

- Watts Humphrey argued that system development problems must be addressed by treating development as a process that can be controlled, measured, and improved.
- The Personal Software Process (PSP) emphasized disciplined collection of individual-level data across the full development lifecycle for continuous improvement.
- Core PSP measures included time by phase, size such as lines of code, defect counts, defect density, and yield from reviews and inspections.
- **Introspective approach:** Users develop analysis routines to evaluate their own process.
- The Team Software Process (TSP) implemented this at the team level.

PSP and TSP Methodologies (2/2)

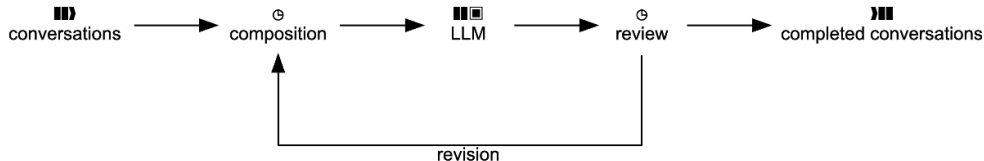
- The PSP and TSP also emphasized defect categorization, enabling engineers to understand not only how many defects occurred, but what kinds occurred and where they were injected and removed.
- This supported phase-based analysis of performance, such as planning accuracy, engineering effort, testing effectiveness, and rework burden.
- Data collection was not an end in itself, but the basis for estimation, process control, quality improvement, and professional discipline.
- PGP is inspired by the PSP to support systems engineering process improvement with empirical data collection and simulation.
- PGP similarly frames generative AI usage as a measurable activity where the user is both the operator and the analyst.

Analogous Metrics for Human-AI Processes

- Defining metric indicators for composition, generation, review, revision, and acceptance cycles.
- At the personal level, metrics include prompt composition time, revision time, review time, number of cycles, acceptance probability, total elapsed time, and output success rate.
- Additional introspection measures can include prompt size, degree of restructuring, micro-edit frequency, review burden, and categories of AI output defects or deficiencies.
- Examples of AI-oriented defect categories include factual error, omission, ambiguity, structural noncompliance, weak reasoning, formatting error, and insufficient task alignment.
- At the team level, analogous measures support a Team Generation Process (TGP) with shared prompt assets, reviewer roles, integration checkpoints, defect classification practices, and comparison across models and workflows.

Generative AI Process Control Loop with Phases

- **Conversations:** Human user initiations of goal-oriented tasks consisting of prompts and responses.
- **Composition:** User formulates/revises prompts using domain knowledge.
- **LLM Invocation:** Stochastic processing time for model output.
- **Review:** User inspects LLM output against task objectives and standards.
- **Revision:** Narrowing scope or adding constraints with another iteration if output is unsatisfactory.

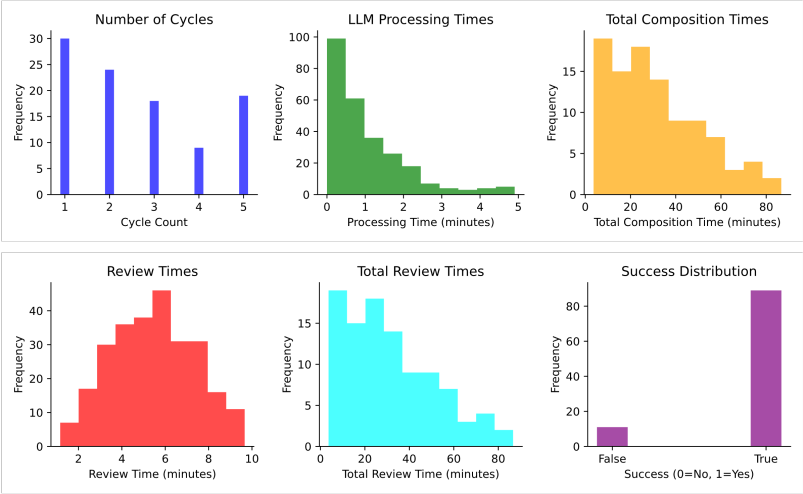


Simulation Implementation via se-lib

- Implemented in Python using the Systems Engineering Library (se-lib).
- Discrete event simulation with Monte Carlo analysis for stochastic behavior.
- Task durations modeled with initial probability distributions.
- Task sizes are calibrated to student level homework problems in systems modeling.

```
# Personal Generation Process Model
import selib as se
se.init_de_model()
se.add_source('conversations', entity_name="conversation",
              num_entities=1, connections={'composition': 1},
              interarrival_time=0)
se.add_delay(name='composition',
             delay_time='random.triangular(1,5,30)',
             connections={'LLM': 1})
se.add_server(name='LLM', connections={'review': 1},
              service_time='random.uniform(1,2)', capacity=1)
se.add_delay(name='review',
             delay_time='random.triangular(1,10,5)',
             connections={'completed prompts': .5, 'revision': .5})
...
```

Monte Carlo Simulation Results Example



Conclusions and Initial Findings

- Established a quantitative foundation for human–LLM interaction dynamics.
- Composition and review times dominate the total elapsed cycle time.
- LLM processing latency is negligible compared to human cognitive effort.
- Process variability is driven primarily by human decision-making.
- Baseline experiments show an overall success rate of approximately 85% within 5 iterations.
- Most tasks complete in few iterations, but 'long-tail' cases present schedule risk.
- Long-tail cases represent tasks exceeding expected effort allocations.

Future Work: Organizational Processes

- Single-user PGP is extensible to team-based workflows (analogous to TSP).
- TGP includes shared prompt libraries and defined reviewer/integrator roles.
- Models how shared repositories influence throughput in large projects.
- PGP loops can be embedded in larger systems engineering lifecycle models.
- Enables project managers to estimate aggregate impact of AI on cost and schedule.
- Supports predictable and auditable integration of AI tools.

Future Work: Interaction Modality

- Extend the model to represent alternative composition and editing modalities, especially typing versus voice dictation.
- Examine how input modality affects prompt entry speed, micro-editing effort, review burden, and total cycle time.
- Analyze the tradeoff between rapid idea capture and precise local control during iterative prompting.
- Measure when voice interaction improves throughput and when typing remains superior for exact wording and structured refinement.
- Use empirical logging of composition, revision, and review behavior to calibrate modality specific parameter distributions.

Future Work: Comparative LLM Performance

- Extend the model to compare alternative LLMs as different service configurations within the same human–AI process.
- Leverage current research on selecting, benchmarking, and deploying LLMs for systems engineering use.
- Represent differences in effectiveness, latency, consistency, review burden, and model size as explicit parameters.
- Examine how model choice influences acceptance probability, number of cycles, and total elapsed time.
- Consider practical constraints that vary by model, including memory footprint, compute demands, deployment feasibility, and local execution limits.
- Use comparative experiments to determine which model and workflow combinations are most effective and feasible for different systems engineering tasks.

References

- [1] Madachy RJ. *Software Process Dynamics*. Wiley-IEEE Press; 2008.
- [2] Humphrey WS. *A Discipline for Software Engineering*. Addison-Wesley; 1995.
- [3] Humphrey WS. *Introduction to the Team Software Process*. Addison-Wesley; 2000.
- [4] se-lib development team. *se-lib: Systems Engineering Library (Python package)*. 2023–2026. Available from: <https://pypi.org/project/se-lib/>